

LLM-based Compatibility Detection

xxxxxxx[1]

1 Introduction

2 Background

2.1 API Compatibility Detection

2.2 API Compatibility Type

2.3 Motivating Example

3 Approach

3.1 CoT

3.2 Multi-Agent

4 Evaluation

4.1 Dataset Construction

Dataset A.

Dataset B.

4.2 Experiment Setup

- **Accuracy Evaluation (RQ1):** How is the overall accuracy of `TOOL` compared with the state-of-the-arts?
- **Effectiveness Evaluation (RQ2):** How is the effectiveness of `TOOL` in different API compatibility types?
- **LLM Generalizability (RQ3):** How is the accuracy of `TOOL` using different LLMs?
- **Ablation Study (RQ4):**

Baselines. We selected XXX [?], XXX [?], and XXX [?] as the comparison.

Metrics. We use precision and recall as metrics, which are defined as follows:

4.3 Results

4.3.1 *Accuracy Evaluation (RQ1).*

4.3.2 *RQ2.*

4.3.3 *RQ3.*

4.3.4 *RQ4.*

5 Related Work

5.1 Traditional Compatibility Detection Methods

API compatibility of Python third-party libraries is a critical concern given their widespread use in diverse domains, and relevant detection methods have been a focus of research. SLamothe et al. (2021) conducted a systematic review of

Author's Contact Information:

Manuscript submitted to ACM

API evolution literature, summarizing research trends, state-of-the-art methods, and remaining challenges. For Python, Quan et al. (2022) proposed PyMevo, a static analysis tool that builds a history-sensitive alias-aware API explorer tree to characterize method evolution. Zhang et al. (2024) [?] explored variadic parameter pitfalls in Python, uncovering their potential API compatibility risks. In deep learning, Zhao et al. developed a knowledge-based approach using Stack Overflow-derived knowledge graphs to detect version incompatibilities. For Android, Liu et al. compared nine tools for API-induced compatibility issues, assessing their replicability and effectiveness. Xie et al. addressed cross-repository compatibility with Hera, which builds an offline database to predict, detect, and fix issues across Python and Ubuntu repositories. However, a notable gap exists in these studies: they primarily focus on compatibility issues in manually written code, with limited exploration of how API evolution impacts code generated by automated tools (e.g., LLM-based code completion systems). In this paper, our aim is to address this gap by investigating incompatible API usages in LLM-generated code and developing targeted mitigation strategies.

5.2 LLMs in the Context of API Evolution

Research on LLMs' reasoning enhancement and practical task adaptation has advanced, with relevance to API evolution-related scenarios. Wei et al. (2022) laid the foundation of Chain-of-Thought (CoT) prompting, enabling LLMs to conduct multi-step reasoning—critical for handling complex logic in API evolution. Li et al. (2025) proposed Structured CoT (SCoT) prompting, integrating sequential, branch, and loop structures to boost code generation accuracy, aiding code adaptation in evolving environments. Zhang et al. (2025) optimized CoT with reasoning patterns (Pattern-CoT), selecting diverse demonstrations to enhance LLMs' robustness in varied reasoning scenarios. Liu et al. (2024) presented ERA-CoT, using entity relationship analysis to strengthen LLMs' understanding of relational changes, useful for tracking associated modifications in evolution. For vulnerability detection linked to evolution, Ma et al. (2024) developed iAudit (combining fine-tuning and LLM agents) for smart contract auditing, and Sun et al. (2024) proposed GPTScan (fusing GPT with static analysis) for logic vulnerability detection. Stechly et al. (2024) noted CoT's limitation in planning—reliance on task-specific prompts, which affects adaptability in dynamic evolution scenarios. Notably, none design prompts for API compatibility or prioritize Python APIs. We address this by crafting API compatibility-focused prompts (integrating Python API version traits/parameter changes) and optimizing for Python's dynamic features (e.g., variadic parameters) to align LLMs with Python API evolution.

References

- [1] Vahid Garousi and Mika V Mäntylä. 2016. A systematic literature review of literature reviews in software testing. *Information and Software Technology* 80 (2016), 195–216.